TD 3 : SQL

Fonctions d'agrégation, requêtes imbriquées

2026-10-02

L3 MIASHS
Université Paris Cité
Année 2025
Course Homepage
Moodle

Objectifs

- requêtes imbriquées,
- fonctions d'agrégation.

Prérequis

- schéma `pagila`,
- cours Requêtes imbriquées, CTEs.
- cours Aggrégation, Partitions, Fenêtres.

> ! Les questions qui suivent concernent la base `pagila`. Il est donc nécessaire de changer le `search_path` pour inclure le schéma `pagila`.

Questions

1. Quels sont les films (identifiant et titre) de la catégorie `animation` ? (66 lignes)
2. Quelles sont les films (identifiant et titre) qui sont disponibles dans deux magasins différents, c'est-à-dire présents dans deux inventaires différents ? Proposer **deux versions**, une première avec une requête imbriquée et une seconde sans sous-requête. (563 lignes)
3. Donner le nombre de films dans la base ? (1000)
4. Donner le nombre d'actrices ou d'acteurs par film (identifiant et titre) ? Attention à conserver les films sans acteurs (il y en a 3). Ordonner la liste par ordre décroissant du nombre d'acteurs. La première ligne est `508 LAMBS CINCINATTI  15`
5. Donner la liste des films (identifiant et titre) où le nombre d'actrices ou d'acteurs est au moins de 10 ? (48 lignes) Ordonner la liste par ordre décroissant du nombre d'acteurs.
6. Donner les nombres moyen, minimum et maximum d'actrices ou d'acteurs par film ? Attention à prendre en compte les films sans acteurs. Arrondir la moyenne à 1 chiffre après la virgule. (Résultat : 5.5, 0, 15)
7. Donner trois actrices ou acteurs (identifiant, nom et prénom) qui ont joué dans le plus grand nombre de films ? (identifiants 107, 102, 198 et il y a unicité du résultat)
8. Donner la liste des actrices/acteurs (identifiant, nom et prénom) qui ont joué dans au plus 20 films. (19 lignes) Ordonner la liste par ordre décroissant du nombre de films.

9. Ecrire une requête qui calcule pour chaque film, le nombre de locations et le nombre d'actrices ou d'acteurs. Comment rechercher une éventuelle corrélation entre ces deux variables ? (Pour le film d'identifiant 1, le nombre de locations est 23 et le nombre d'acteurs est 10). Attention à ne pas oublier les films sans location et les films sans acteurs.
10. Quels sont les actrices ou acteurs (identifiant, nom et prénom) qui ont joué plus que la moyenne des actrices/acteurs, ordonnés par le nombre décroissant de films ? (94 lignes). Proposer **deux versions**, une première avec une CTE et une requête imbriquée et une seconde, avec des CTEs et une jointure. Pourquoi la seconde version est-elle plus performante ?
11. Quels sont les acteurs (identifiant, nom et prénom) qui ont joué dans toutes les catégories de film ? (11 lignes) Proposer une solution utilisant les fonctions d'agrégation puis, si possible, une solution sans fonction d'agrégation.
12. Bonus : existe-t-il des acteurs qui n'ont joué avec aucun autre acteur ? (Réponse : non)

Pour aller plus loin : les vues

Les *vues* permettent de donner un nom à une requête afin de pouvoir l'appeler plus tard sans la réécrire à chaque fois. Une vue s'enregistre dans un schéma. Par exemple, dans le schéma `World`, on pourrait créer une vue `VillesRepublic` qui contiendrait toutes les villes de la table `city` qui sont dans une république.

On crée une vue avec `CREATE VIEW nom AS requete`. Étant donné que vous ne pouvez écrire que dans votre schéma personnel, il faudra nommer vos vues `entid.nom` où `entid` est votre identifiant en salle de TP.

```
CREATE VIEW entid.VillesRepublic AS
   SELECT
      ci.*
   FROM
      world.country as co JOIN
      world.city as ci USING (countrycode)
   WHERE
      co.governmentform like '%Republic%';
```

Cette requête crée une vue dans votre schéma personnel. Désormais, si on veut sélectionner les villes qui sont dans une république et dont la population est supérieure à 1000000, on pourra simplement écrire :

```
SELECT *
FROM
    entid.VillesRepublic
WHERE
    population>=1000000;
```

> Remarquez la différence entre `WITH` et une vue. `WITH` nomme une requête temporairement, seulement à l'échelle de la requête courante tandis qu'une vue est enregistrée de façon permanente. Cependant, chaque fois que vous appelez votre vue, elle est réévaluée par le système de gestion de base de données.

Notez aussi que SQL n'est pas sensible à la casse. La vue `entid.VillesRepublic` peut être aussi désignée par `entid.villesrepublic`.

Pour supprimer une vue existante, on utilise la commande `DROP VIEW` suivie du nom de la vue à supprimer. Par exemple l'instruction

```
DROP VIEW entid.VillesRepublic ;
```

supprime la vue créée précédemment.

13. **QUESTION** Dans votre schéma personnel `entid` (il faut remplacer `entid` par votre nom de login), écrire une vue `film_id_horror` qui renvoie la liste des films de catégorie 'Horror'. Puis avec cette vue, écrire une requête qui affiche les titres des films de cette catégorie. (56 lignes)